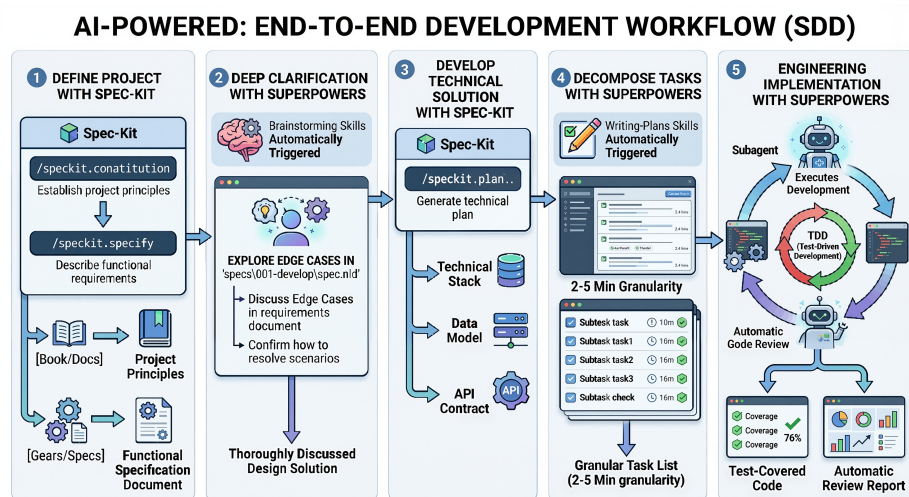


SKILL

Superspec

Superspec unifies spec-kit specification-driven development with obra/superpowers agent capabilities into a single workflow. Spec-kit provides the document structure and governance; superpowers provides deep clarification, task decomposition, and engineering execution discipline.



AI-Powered: End-to-End Development Workflow (SDD)

Prerequisites

Required: None. Superspec works standalone with built-in fallback protocols.

Optional (enhanced): Install obra/superpowers skills to `~/.agents/skills/` or `.agents/skills/` for richer brainstorming, planning, and execution capabilities. See superpowers-bridge.md for detection and integration details.

Project Structure

When initialized, superspec relies on spec-kit's two top-level directories at the project root: `.specify/` for tool metadata and `specs/` for feature artifacts.

```
your-project/
├── .specify/
│   ├── memory/
│   │   └── constitution.md      # Project governance principles
│   └── superpowers.yml        # Superpowers detection status
├── (auto-managed)
│   └── templates/             # Document templates (copied on
init)
└── specs/
    ├── NNN-feature-name/
    │   ├── spec.md             # Feature specification
    │   ├── plan.md             # Implementation plan
    │   ├── tasks.md            # Task breakdown
    │   └── progress.yml        # Phase progress tracker (auto-
managed)
    └── checklist-*.md          # Generated checklists
```

Commands

Command	Purpose
<code>/speckit.superspec.status</code>	Show current progress and suggest next step (resumable)
<code>/speckit.constitution</code>	Create or update project governance principles
<code>/speckit.specify</code>	Create a feature specification with user stories
<code>/speckit.superspec.brainstorm</code>	Deep-dive edge cases and refine a spec document
<code>/speckit.plan</code>	Create a technical implementation plan
<code>/speckit.superspec.tasks</code>	Generate a phased task breakdown
<code>/speckit.superspec.execute</code>	Orchestrate implementation with TDD + subagents
<code>/speckit.superspec.review</code>	Run code review against spec requirements
<code>/speckit.checklist</code>	Generate a contextual checklist

Session Resumability

Superspec is designed to be **fully resumable across sessions**. All state is persisted in the `.specify/` directory as markdown files. When a session is interrupted (agent timeout, user leaves, CLI crash), no progress is lost.

Progress Tracking

Each feature spec directory contains a `progress.yml` file that records phase status:

```
# specs/NNN-feature-name/progress.yml
feature: feature-name
created: 2026-04-22
current_phase: brainstorm
phases:
  constitution: { status: done, updated: 2026-04-22 }
  specify:      { status: done, updated: 2026-04-22 }
  brainstorm:   { status: in_progress, updated: 2026-04-22,
                sessions: 1 }
  plan:         { status: pending }
  tasks:        { status: pending }
  execute:      { status: pending }
  review:       { status: pending }
```

Status values: pending, in_progress, done, skipped

Every command updates `progress.yml` when it starts (in_progress) and finishes (done).

Superpowers Status Tracking

A project-level file `.specify/superpowers.yml` records which superpowers skills are available. This makes the superpowers integration **visible in the project docs** and **persistent across sessions** — no need to re-detect on every command.

```
# .specify/superpowers.yml
last_checked: 2026-04-22T14:30:00
skills:
  brainstorming:
    detected: true
    path: ~/.agents/skills/brainstorming/SKILL.md
  writing-plans:
    detected: true
    path: ~/.agents/skills/writing-plans/SKILL.md
  executing-plans:
    detected: false
  subagent-driven-development:
    detected: false
```

```
test-driven-development:
  detected: true
  path: .agents/skills/test-driven-development/SKILL.md
requesting-code-review:
  detected: false
```

When this file is updated: - On /speckit.constitution (initial creation) - On /speckit.superspec.status (re-check) - On any command that needs a superpowers skill (lazy re-check if missing) - User can manually edit this file to override detection results

Why persist this: So that project documentation reflects which superpowers are in use. A new team member reading .specify/ can immediately see the project's enhanced capabilities without running any command.

Resume Protocol

When ANY superspec command is invoked, the agent **MUST** first run the **resume check**:

1. Check if .specify/ directory exists
2. If yes, scan for progress.yml files in each spec directory
3. Read the most recent progress.yml to determine current_phase
4. Read .specify/superpowers.yml to determine which superpowers are available. If the file does not exist, run superpowers detection and create it.
5. Report to user: "Detected existing progress for [feature]: [phase] is [status]. Superpowers: [list detected skills]. Resuming from this point." or "No previous progress found, starting fresh."
6. For in_progress phases: re-read all existing artifacts for that phase and continue where the agent left off (e.g., resume brainstorming from the last logged session, resume execution from the first unchecked task)

How Each Phase Resumes

Phase	Resume Signal	Resume Behavior
constitution	constitution.md exists but incomplete	Re-read and ask about missing sections
specify	spec.md exists with [NEEDS CLARIFICATION] markers	Continue interview for unresolved items
brainstorm	Brainstorm Log has entries, Open Questions has Open items	Skip already-explored categories, continue from open questions
plan		Fill in missing technical context

Phase	Resume Signal	Resume Behavior
	plan.md exists with NEEDS CLARIFICATION fields	
tasks	tasks.md exists	Verify completeness, add missing tasks
execute	tasks.md has mix of [x] and [] checkboxes	Skip completed tasks, resume from first unchecked task in current phase
review	Review checklist partially completed	Continue from unchecked review items

/speckit.superspec.status

Input: Optional spec number or “all” via \$ARGUMENTS. Defaults to showing all features. **Output:** Progress report printed to user.

Process: 1. Scan .specify/ directory structure 2. Check if constitution.md exists 3. **Run superpowers detection:** Check for all superpowers skills at .agents/skills/ and ~/.agents/skills/. Update .specify/superpowers.yml with current detection results. 4. For each spec directory, read progress.yml (or infer progress from existing files) 5. Display a status summary:

Superspec Project Status

=====

Constitution: Done (2026-04-22)

Superpowers: brainstorming (detected), writing-plans (not found)

Features:

001-user-auth [####-----] execute (Phase 5/6) – T012/T019
tasks done

002-photo-upload [##-----] brainstorm (Phase 2/6) – 2 open
questions

003-settings [#-----] specify (Phase 1/6) – draft

Suggested next step: /speckit.superspec.execute 001

1. If no .specify/ exists, suggest: “No superspec project found.
Run /speckit.constitution to get started.”

File inference fallback: If progress.yml does not exist, infer progress from which files are present: - spec.md exists → specify is done - spec.md has Brainstorm Log entries → brainstorm was run -

plan.md exists → plan is done - tasks.md exists → tasks are done - tasks.md has [x] checkboxes → execute is in progress (count checked vs total)

/speckit.constitution

Input: Project name and optional description via \$ARGUMENTS.

Output: .specify/memory/constitution.md

Process: 1. Create .specify/ directory structure if it does not exist
2. Copy all files from this skill's templates/ directory into .specify/templates/
3. Read the template at .specify/templates/constitution-template.md
4. Interview the user about core principles, technology stack, design system, quality gates
5. Generate constitution.md from template, filling in user responses
6. Write to .specify/memory/constitution.md

Gate: Constitution must exist before any other command can run.

/speckit.specify

Input: Feature name and description via \$ARGUMENTS. **Output:** specs/NNN-feature-name/spec.md

Process: 1. Verify .specify/memory/constitution.md exists (abort with guidance if not)
2. Determine the next spec number NNN (scan existing specs/ directories)
3. Read the template at .specify/templates/spec-template.md
4. Read the constitution to understand project principles and constraints
5. Interview the user about user scenarios, requirements, success criteria
6. Generate spec.md from template with user responses
7. Write to specs/NNN-feature-name/spec.md

Next step suggestion: Run /speckit.superspec.brainstorm on the new spec to discover edge cases.

/speckit.superspec.brainstorm

Input: Path to a spec file (e.g., specs/001-auth/spec.md) and an optional focus topic via \$ARGUMENTS. **Output:** Updated spec file with refined edge cases, resolved open questions, and brainstorm log entries.

Process: 1. Read the target spec file 2. Read the constitution for project constraints 3. **Superpowers detection:** Check for brainstorming skill (see [superpowers-bridge.md](#)) - **If found:** Read the brainstorming SKILL.md and follow its questioning protocol, adapting all outputs to the target spec file - **If not found:** Use the built-in questioning protocol (see [workflow-guide.md](#) Phase 2) 4. Ask questions **one at a time**, focusing on: - Boundary conditions and edge cases - Error scenarios and failure modes - Scale and performance implications - Security and privacy concerns - User confusion and UX pitfalls 5. After each answer, update the spec's "Open Questions" section (mark resolved items) 6. When the user confirms the spec is ready, update the "Brainstorm Log" with a dated summary of insights discovered

Iteration: This command can be run multiple times on the same spec. Each session appends to the brainstorm log.

/speckit.plan

Input: Optional spec number or path via \$ARGUMENTS. Defaults to the latest spec. **Output:** specs/NNN-feature-name/plan.md

Process: 1. Read the target spec file and constitution 2. Read the template at .specify/templates/plan-template.md 3. Perform a **constitution check** — verify the plan aligns with all governance principles 4. Research the codebase to determine technical context (language, dependencies, storage, testing framework, project type) 5. Design the project structure and identify files to create or modify 6. Determine the **execution strategy**: which tasks need TDD, which support parallel subagent execution, where human checkpoints are needed 7. Generate plan.md from template 8. Write to specs/NNN-feature-name/plan.md

Superpowers bridge: If writing-plans skill is detected, read it and use its blueprint generation process to enhance the plan's task structure section. See [superpowers-bridge.md](#).

/speckit.superspec.tasks

Input: Optional spec number or path via \$ARGUMENTS. Defaults to the latest spec. **Output:** specs/NNN-feature-name/tasks.md

Process: 1. Read the spec, plan, and constitution for the target feature 2. Read the template at .specify/templates/tasks-template.md 3. **Superpowers detection:** Check for writing-plans skill - **If found:** Read the writing-plans SKILL.md and follow its task decomposition process, adapting outputs to the tasks

template structure - **If not found:** Decompose directly from the plan using the template 4. Organize tasks by phase: Setup → Foundational → User Stories (by priority) → Polish 5. Apply execution markers to each task: - [P] — can run in parallel (different files, no dependencies) - [TDD] — must follow RED-GREEN-REFACTOR discipline - [REVIEW] — requires code review before proceeding - [SUBAGENT] — can be delegated to a subagent 6. Define phase dependencies and checkpoint gates 7. Write to specs/NNN-feature-name/tasks.md

/speckit.superspec.execute

Input: Optional spec number or path via \$ARGUMENTS. Defaults to the latest spec. **Output:** Code changes in the project, updated task checkboxes.

Process: 1. Read the tasks file for the target feature 2. Read the plan and constitution for context 3. **Superpowers detection:** Check for executing-plans, subagent-driven-development, and test-driven-development skills 4. Walk through tasks phase by phase: - **[TDD] tasks:** If TDD skill found, follow its RED-GREEN-REFACTOR process. Otherwise: write test → verify it fails → implement → verify it passes - **[SUBAGENT] tasks:** If subagent-driven-development skill found, follow its dispatch protocol. Otherwise: implement sequentially in-session - **[P] tasks:** Launch parallel tasks where possible using the Task tool - **[REVIEW] tasks:** Pause and run review protocol (see /speckit.superspec.review) 5. At each **phase checkpoint:** Summarize completed work, run tests if applicable, ask user for approval before proceeding to next phase 6. Update task checkboxes in tasks.md as each task completes

Human checkpoints: The agent MUST pause at every phase boundary and wait for explicit user approval. Never skip a checkpoint.

/speckit.superspec.review

Input: Optional scope (file paths or “all changes”) via \$ARGUMENTS. **Output:** Review findings reported to user, optionally written to a checklist file.

Process: 1. Read the spec and plan for the feature being reviewed 2. **Superpowers detection:** Check for requesting-code-review skill - **If found:** Read the skill and follow its pre-evaluation checklist and review dispatch protocol - **If not found:** Use the built-in review protocol below 3. Built-in review protocol: - **Spec compliance:** Verify each acceptance scenario from the spec is

implemented - **Edge case coverage:** Verify brainstormed edge cases are handled - **Constitution compliance:** Check all governance principles are respected - **Code quality:** Check for bugs, security issues, error handling - **Test coverage:** Verify tests exist for critical paths 4. Report findings with confidence scores (0-100, only report issues ≥ 80) 5. Group findings by severity: Critical > Important > Suggestion

/speckit.checklist

Input: Checklist type and optional context via \$ARGUMENTS. **Output:** specs/NNN-feature-name/checklist-{type}.md

Process: 1. Read the template at .specify/templates/checklist-template.md 2. Read the spec, plan, and tasks for context 3. Generate a checklist appropriate to the requested type (e.g., “launch readiness”, “security audit”, “accessibility review”, “code review”) 4. Write to specs/NNN-feature-name/checklist-{type}.md

Unified Workflow

The recommended end-to-end workflow:

Phase 0: /speckit.constitution	→ Establish project governance
Phase 1: /speckit.specify	→ Define feature requirements
Phase 2: /speckit.superspec.brainstorm	→ Clarify edge cases (iterate)
Phase 3: /speckit.plan	→ Design technical approach
Phase 4: /speckit.superspec.tasks	→ Decompose into executable tasks
Phase 5: /speckit.superspec.execute + subagents	→ Implement with TDD
Phase 6: /speckit.superspec.review	→ Verify against spec

Each phase has an explicit **gate** — the agent verifies prerequisites before proceeding. Run /speckit.superspec.brainstorm multiple times until the spec is solid. The user controls when to advance to the next phase.

Additional Resources

- Detailed phase-by-phase guide: [workflow-guide.md](#)
- Superpowers integration details: [superpowers-bridge.md](#)
- End-to-end example: [sample-workflow.md](#)